티칭로그 AI 서버 PRD

AI 학생을 가르치며 배우는 학습 서비스 — AI 파트 개발 요구사항

문서	제품 요구사항 문서 (PRD) · AI 서버 범위
버전 · 날짜	v1.14 · 2026-11-11
기준 코드	`dev` 브랜치 최신 + 팀 결정
기준 문서	기능명세서, AI 서버 API 명세서(AI-0 ~ AI-9), 백엔드 API 명세서(01 ~ 24), ERD, 로그 규칙. 충돌하면 이 문서들이 우선한다. 명세에 아직 없는 동작은 코드 로 표시했다 (명세에 반영 필요).
변경 이력	**v1.14** (2026-11-11): AI-5 빈 용어 문구 변경, 순서만 알리는 말 제외, "!," 정리, 실측 갱신 v1.13: 정리노트를 체크리스트 용어별(개념 · 학생 AI의 질문 · 확인하지 않은 질문)로 바꿈. AI-5 요청·응답에 `terms` 추가. 나의 답변은 선생님 답 원문 그대로. WebP 이미지 지원. 백엔드 수정 사항 8가지 정리 v1.12: 평가 5문제·재시험 3문제 확정(팀 결정 2026-11-11). 10문제 실측 결과와 문제 수를 줄인 이유를 적었다. 질문받기 뒤 첫 선생님 메시지를 답변으로 보는 규칙을 추가했다. v1.11: 짝 개념 실제 모델 확인 결과를 넣었다. v1.10: 짝 개념 규칙(이름 A·B, 한쪽 이름 인정)을 적고 반영 상태로 바꿨다. 첫 설명 예외를 주제 시점으로 좁혔다. 재시험 문제 수를 논의 필요로 바꿨다. v1.9: Django 점검 항목에 Gunicorn 타임아웃을, 평가 진행 화면에 진행 표시 참고를 더했다. v1.8: 백엔드 코드를 대조해 재시험 문제 수 차이(PRD 3문제, 백엔드 10문제)와 그 위험, 유형 배정 방식을 적었다. v1.7: 평가 문제 수를 백엔드 기본값(`EXAM_QUESTION_COUNT` =10)에 맞춰 10문제로 바꿨다. v1.6: 팀 결정을 반영했다. 평가 문제 수·유형, 평가 진행 화면, 업로드 크기, 짝 개념 합치기, 새 페르소나 글투를 확정했다. v1.5: 정리노트를 "개념 설명 · 질문과 답 · 아직 궁금한 사항" 3부분으로 고정했다. AI-3에 학생 반응(`reaction`)을, AI-5에 `raised_reasons` 를 더했다. 페르소나 글투를 AI 서버(`voices.json`)로 옮겼다. v1.4: 최신 코드에 맞췄다. AI-1에서 이미지로 읽은 자료를 AI-2가 다시 쓰는 읽기 캐시, 손 들기의 명사 끝맺음 예외, AI-4 기호 표기 검사를 반영했다. 화면 연동 요구사항(디자인 대조), 기능명세서의 미정 항목, 명세 반영 필요 표시, 정리노트 구역 결정을 더했다. (같은 날 쓴 v2.0 초안은 이전 코드 기준이라 이 문서로 대체한다) v1.3: 코드 현재 상태에 맞췄다. • AI-1은 핵심 용어만 고른다 (3~10개). • AI-3 손 들기는 세 시점에만 판단한다. 한 글자 용어를 매칭하고, 판정과 손 들기 예외를 격리했다. • AI-4는 물음표를 하나만 쓴다. AI-5는 페르소나 말투로 쓰고 선생님 문장 빠짐을 검사한다. AI-7 객관식 답은 숫자 하나다. • 페르소나를 4명으로 늘렸다. 입력 상한과 설정 오타 검사를 추가했다. • 정의 검사가 실패하면 건너뛴다. 잘린 호출의 토큰도 기록한다. 검사 로그는 DEBUG로 내렸다. Render로 배포한다. v1.2: 실제 모델 검증 반영 (인용 8자, 검사 AI 이유 먼저, AI-2 부분 재시도, AI-6 해설·보기 규칙) v1.1: 정의 독립 검사, 본문 검사, 숫자 대조, 질문 용어 검사, AI-2 묶음 호출 · v1.0: 최초 작성
구현 상태	• FastAPI 서버 구현 완료. 실제 Claude로 AI-1 ~ AI-9 확인 (v1.3, 재시험은 v1.0만 확인) • 자동 테스트는 개발용 폴더에 따로 보관한다 (v1.3 기준 pytest 41개 + 점검 5묶음 통과)
모델	모든 기능이 Anthropic `claude-haiku-5-5` 하나를 쓴다
대회	1호선톤 · 주제 「AI로 다시 그리는 학습과 성장」

표시: 결정 원래 명세 뒤에 팀이 정함 · 코드 코드에 이미 있음, 명세에 반영 필요 · 미반영 정했지만 코드에 아직 없음 · 미정 팀이 정해야 함

1. 배경과 목표
2. 범위
3. 사용자 흐름과 AI 호출
4. AI 구성 원칙
5. 기능 요구사항 (AI-0 ~ AI-9)
6. 상태값과 데이터 규칙
7. 품질·할루시네이션 요구사항
8. 비기능 요구사항
9. 비용
10. 설정과 실행
11. 화면 연동 요구사항
12. 검증과 수용 기준
13. 남은 과제
14. 위험과 팀 결정 사항
15. 용어

1. 배경과 목표

문제

문제	설명
안다는 착각	자료를 다시 읽으면 익숙해서 아는 것 같지만, 막상 설명하려면 막힌다.
빈틈을 못 찾음	무엇을 모르는지 몰라서 어디를 복습할지 알기 어렵다.
수동적인 AI 학습	AI에게 물으면 답은 빨리 얻지만, 생각하고 정리하는 과정을 AI가 대신한다.
성장이 안 보임	복습해도 얼마나 나아졌는지 확인하기 어렵다.

해결 방향

- 파인만 학습법(남에게 설명하며 배우기)을 AI로 구현한다. 사용자가 선생님, AI가 학생이다.
- **AI 학생은 사용자가 가르친 것만 안다.** 빠뜨리면 시험에서 "모르겠어요", 틀리게 가르치면 똑같이 틀린다.
- 학생의 시험 결과가 곧 사용자의 설명 실력이자 이해의 빈틈이다. 성장보고서와 재시험으로 성장을 숫자로 보여 준다.

AI 파트 목표

#	목표	측정 기준
G1	자료에 근거한 판단	근거 문장이 자료나 대화에 실제로 있다(원문 인용 검증). 본문도 그 근거와 같은 말이다(정의 독립 검사, 본문 검사, 숫자 대조).
G2	학생 AI의 정보 분리	학생 AI 입력에 자료, 용어 정의, 요약정리, 정답이 들어가지 않는다.
G3	명세 그대로의 연동	요청·응답 필드, 에러 code 5개, 로그 이벤트가 API 명세·로그 규칙과 같다. 명세에 없는 동작은 `코드` 로 표시하고 명세에 반영한다(13장).
G4	시간 안에 응답	실시간 API는 25초, 백그라운드 API는 170초 안에 끝나거나 정해진 에러를 준다. v1.2 실측은 모두 예산 안이다.
G5	낮은 비용	전 기능 Haiku 5.5. 수업 1회 비용은 v1.0에서 수십 원 수준이었다. 검사 호출은 늘었고, 손들기 호출과 자료 다시 읽기는 줄었다. 재측정이 필요하다.

2. 범위

AI 서버가 한다	AI 서버가 하지 않는다 (Django·프론트)

• 자료 읽기(PDF·텍스트·코드·이미지), 용어 체크리스트, 요약정리 • 수업 중 용어 판정, 손 들기 판단, 학생 질문 만들기 • 정리노트, 출제, 학생 AI 문제 풀이, 주관식 채점 • 성장보고서 문장 (잘한 점, 복습 추천)	• 로그인, DB 저장, 수업·시험 상태 관리, 백그라운드 작업 • PPT·DOCX → PDF 변환, 원본 삭제 • 점수·비교 계산, 객관식 채점(보기 인덱스 비교), 놓친 개념 목록 • 손 든 표시 저장·노출, 첫 인사(페르소나별 고정 문장) • 학생 반응 말풍선 표시 (AI-3 `reaction`을 보여 주되 messages에는 저장하지 않음)

- **호출 주체**: AI 서버는 Django만 부른다.
- **상태 없음**: AI 서버는 DB를 쓰지 않는다. 필요한 정보는 요청마다 Django가 보낸다.
  - 업로드 처리(PDF를 이미지로 바꾸기, 이미지 줄이기)는 메모리에서 한다.
  - 1MB가 넘는 업로드는 프레임워크(Starlette)가 받는 동안 임시 파일에 두었다가, 요청이 끝나면 지운다.
- **읽기 캐시** 코드 : AI-1에서 AI가 이미지로 읽은 쪽 글자만 메모리에 둔다(파일 내용의 SHA-256이 열쇠).
  - 최대 15분, 16건까지 둔다. AI-2가 성공하면 바로 지운다.
  - 디스크에는 쓰지 않는다. 서버를 재시작하면 사라지고, 프로세스마다 따로다.
  - API 형식은 그대로다(AI-1·AI-2는 여전히 같은 파일을 따로 받는다).

```
프론트엔드 ──▶ Django (/api) ──(X-Internal-Token, X-Request-ID)──▶ AI 서버 (FastAPI) ──▶ Anthropic Claude
```

3. 사용자 흐름과 AI 호출

#	화면 · 사용자 단계	백엔드 API	AI 호출	AI가 하는 일
1	자료 제출: 제목 + 파일/텍스트, 학생 선택	08 → 09 폴링	AI-1 → AI-2 (백그라운드)	용어 체크리스트를 만들고, 그 용어로 요약정리를 만든다. 약 2~3분 걸린다.
2	수업 시작	10	-	첫 인사는 Django가 고정 문장으로 만든다.
3	설명·답변 보내기	14	AI-3 (실시간)	용어 상태를 판정하고, 손을 들지 판단한다.
4	손 든 표시 / 질문 받기 (n)	15	AI-4 (실시간)	손 든 이유로 학생 질문을 하나 만든다.
5	수업 종료	16 → 17 폴링	AI-5 → AI-6 → AI-7 → AI-8 → AI-9	정리노트, 평가 출제, 학생 AI 풀이, 주관식 채점, 성장보고서를 만든다.
6	정리노트·성장보고서 보기	18, 20	-	-
7	재시험 (사용자가 풂)	21 → 22, 23, 24	AI-6, AI-8 (문항마다)	놓친 용어 위주로 출제하고, 주관식은 한 문제씩 채점한다.

평가는 **학생 AI**가 풀고(AI-7), 재시험은 **사용자**가 푼다. 자료 하나로 수업은 한 번만 할 수 있다(`409 MATERIAL_ALREADY_USED`). "같은 자료로 다시 수업"하는 기능은 없고, 끝나지 않은 수업만 이어서 할 수 있다.

4. AI 구성 원칙

4-1. 두 AI, 보는 정보를 분리

구분	평가 AI (material · judge · evaluator · report)	학생 AI (student)

역할	용어·요약정리, 용어 판정, 출제·정답 검사, 채점, 보고서 문장	손 들기, 질문, 정리노트, 평가 시험 풀이
보는 정보	자료, 요약정리, 용어 목록, 대화	페르소나 + **수업 대화** + 손 든 이유 메모
금지	학생처럼 말하기	자료, 용어 정의, 요약정리, 정답 보기. 원래 지식으로 빈칸 채우기

4-2. 판단은 AI, 확정은 코드

- AI는 언어 이해가 필요한 판단만 한다: 핵심 용어 고르기, 설명했는지, 손을 들지, 주관식 답이 맞는지, 어떤 문장을 쓸지.
- 모든 페르소나 프롬프트 뒤에는 공통 학생 규칙(`student_rules.md`)이 붙는다: 배운 것만 안다, 질문은 하나, 정답을 미리 말하지 않는다.
- 코드가 정하는 것
  - 요약정리·정리노트 Markdown 조립
  - 출제 계획(유형 순서, 용어 배정)과 보기 섞기
  - 용어 상태는 올라가기만 하게 함
  - 빈 답은 LLM 없이 오답 처리
  - 문항 번호·점수 검증, 손 들기 판단 시점
  - 질문 짝 짓기와 용어별 묶기, 정리노트 화면용 구조 조립, 학생 반응 문장 고르기

4-3. 모델

모든 기능이 `claude-haiku-5-5` 하나를 쓴다. 기능마다 모델을 나누지 않는다.

AI-3 용어 판정만 같은 모델에 생각 깊이(effort) `high`를 붙인다. 다른 모델이 아니라 단가는 같고, 생각에 쓴 토큰만큼 비용·시간이 조금 는다. 나머지는 모델 기본(`medium`)이다.

4-4. 학생 페르소나 결정 4명

code	이름	학년	비고
`haneul`	하늘	초5	말투·질문 수준·배경지식은 `app/personas/personas_seed.json`의 `system_prompt`에 있다. Django 시드로 넣는다. v1.3에서 예시 문장이 바뀌어 다시 시드해야 한다. 4명째 추가에 맞춰 ERD의 "3명 고정" 설명, 07번 응답, 10번 첫 인사 문장(교수님용)을 백엔드가 고쳐야 한다 미반영.
`minjun`	민준	고1	
`seoyeon`	서연	대2	글투(말하는 방식)는 AI 서버의 `app/personas/voices.json`에 있다. 질문(AI-4), 정리노트(AI-5), 시험 주관식(AI-7)에 붙는다. 글투가 바뀌어도 Django는 다시 시드할 필요가 없다. 단, Django에서 새로 만든 페르소나는 글투가 없어서 기본 말투로 쓴다.
`hyunwoo`	정현우 교수님	교수	

5. 기능 요구사항

공통: 모든 요청 헤더에 `X-Internal-Token`, `X-Request-ID`가 들어가고, 모든 응답에 `usage[]`와 응답 헤더 `X-Request-ID`가 들어간다. 필드 이름은 API 명세와 같다.

ID	주소	역할	시간 예산
AI-0	`GET /health`	-	- (인증 없음)
AI-1	`POST /materials/terms`	material	170초

AI-2	`POST /materials/summary`	material	170초
AI-3	`POST /lessons/messages/analyze`	judge + student	25초
AI-4	`POST /lessons/questions`	student	25초
AI-5	`POST /lessons/note`	student	170초
AI-6	`POST /exams/generate`	evaluator	170초
AI-7	`POST /exams/solve`	student	170초
AI-8	`POST /exams/grade`	evaluator	1문항 25초, 여러 문항 170초
AI-9	`POST /reports/generate`	report	170초

AI-1 용어 체크리스트

입력	multipart: `title`, `file`(PDF·텍스트·코드·PNG·JPG·WebP) 또는 `text` 중 하나. PPT·DOCX는 Django가 PDF로 바꿔 보낸다.
출력	`terms: [{name, definition}]`
규칙	• **개수는 자료 길이로 정하지 않는다.** AI가 핵심 용어만 고르고, 최소 3개 ~ 최대 10개다 (`MIN_TERMS` / `MAX_TERMS`). 150자 미만 자료는 최소 1개다. • 핵심 용어 기준 (셋 다 맞아야 함) ○ ① 자료가 그 용어의 정의나 원리를 직접 설명한다. ○ ② 빼면 수업에 구멍이 생긴다. ○ ③ 다른 고른 용어와 겹치지 않는다. 다른 이름·부분·변형은 하나로 합친다(예: "RNN"과 "RNN 계층"). • 고르지 않는 것: 함수·인수·변수 이름(자료가 중심 개념으로 설명할 때만 예외), 예시 속 사례, 수식 기호, 장·절 제목. • 용어 이름은 **자료에 실제로 나오는 말**이어야 한다. 설명형 이름은 거부한다. 짝 개념은 A나 B가 자료에 나오면 된다. • **짝 개념**(예: 전역 최솟값·지역 최솟값)은 용어 하나로 합친다. 이름은 `A·B` 형식이고 30자까지다. 정의에는 두 개념과 그 차이를 같이 쓴다. • 정의는 자료 문장 인용으로 검증하고, 근거 청크에 없는 숫자(두 자리 이상·소수)가 있으면 거부한다. • 정의 독립 검사: 별도 호출이 정의가 근거로 뒷받침되는지, 이 용어 자체의 정의인지 다시 본다. ○ 떨어진 용어가 있으면 이유를 붙여 한 번 더 뽑는다. ○ 검사 호출 자체가 실패하면 그 검사만 건너뛴다(DEBUG 기록). ○ 다시 부를 때는 이미 본 정의의 검사 결과를 재사용한다.
수용 기준	RNN 30쪽 PDF에서 핵심 용어 5개 내외(순환 신경망, 은닉 상태, BPTT, Truncated BPTT, 언어 모델)가 나온다. 세부 용어(stateful, Time RNN 계층 등)는 들어가지 않는다. 읽을 수 없는 자료는 `422 UNREADABLE_MATERIAL` 이다.

AI-2 요약정리

입력	multipart: `title`, `terms` (AI-1 결과 JSON 문자열), AI-1과 같은 `file` 또는 `text`
출력	`summary` : Markdown (용어마다 정의와 요점 2~4개, 용어 사이 관계)
규칙	• RAG(BM25)로 용어마다 관련 청크만 보여 준다. 요약은 용어 3개씩 묶어 동시에 부르고, 관계는 따로 한 번 부른다. • 요점은 청크 ID와 원문 인용으로 검증한다. 다른 용어 이야기, 정의 되풀이, 다른 용어에 이미 쓴 예시, 근거 없는 숫자는 뺀다. • 본문 검사: 별도 호출이 요점·관계 문장이 근거 청크만으로 나오는지 본다. 뜻 뒤바뀜, 인과 뒤바뀜, 숫자를 다른 대상에 붙인 경우를 막는다. • 떨어진 요점은 빼고 쓴다. 묶음을 다시 부르는 것은 요점이 2개 미만으로 줄었을 때와 응답이 잘렸을 때뿐이다. • 요점은 2~4개가 목표지만, 근거가 확인되는 만큼만 쓴다(자료가 얇으면 1개도 정상). 요점이 0개인 용어만 한 번 더 요약하고, 그래도 비면 정의 줄만 남긴다. • 코드 AI-1에서 이미지로 읽은 쪽은 읽기 캐시에서 가져온다. 다시 읽지 않으니 30쪽 이미지 PDF 기준으로 약 13초, 호출 8번이 줄고, AI-1과 같은 글자를 근거로 쓴다. • 관계 문장에는 두 용어 이름이 모두 들어 있어야 한다. Markdown은 코드가 조립한다.
수용 기준	모든 요점이 자료 인용으로 뒷받침되고, 출제(AI-6)의 유일한 근거로 쓸 수 있다.

AI-3 용어 판정 + 손 들기

입력	`judge`: 방금 보낸 메시지(`explanation` / `answer`), 답변이면 `reply_to_question`, 용어 목록과 현재 상태 `student`: 페르소나, **방금 보낸 메시지까지 포함한** 전체 대화, 아직 질문하지 않은 `raised_reasons`
출력	`term_updates: [{term_id, status}]`, `hand_raise: {reason} | null`, `reaction: string | null`
판정 규칙 결정	• **용어 이름을 말하고, 간단하게라도 맞게 설명하면** `explained`(체크)다. 정의를 다 말하지 않아도 핵심 하나가 맞으면 된다. 쉬운 말이나 비유도 뜻이 맞으면 인정한다. • 짝 개념(`A·B`)은 A나 B 중 한쪽 이름만 말해도 용어 이름을 말한 것으로 본다. • 이름만 말했거나 틀리게 설명했으면 `mentioned`(체크 안 됨), 안 나왔으면 `todo`다. • 이름은 괄호 별칭도 인정한다(예: "순환 신경망(RNN)"이면 "RNN"도). 답변이면 학생 질문 속 이름도 인정한다. 한 글자 용어("큐", "힙")는 낱말 경계 + 조사로 매칭해서 "큐브", "힙합" 속 글자는 잡지 않는다. • 명세대로 방금 보낸 메시지 하나만 본다. 판정 기준은 AI-1의 정의뿐이고, 근거는 메시지 원문 인용으로 검증한다. 상태는 올라가기만 한다.
손 들기 규칙 결정	• **주제가 바뀔 것 같을 때만 판단**한다. ○ ① 새 용어로 넘어감, 또는 "다음으로 넘어가자", "정리하면" ○ ② "알겠지?", "질문 있어?"처럼 학생에게 차례를 넘김 ○ ③ 학생 질문에 답한 직후 그 밖에는 LLM을 부르지 않고 `hand_raise: null`을 돌려준다. 같은 주제를 이어 설명하는 중, 설명 도중(":", "그리고", "-고/-서/-면"으로 끝남), 짧은 맞장구에서는 판단하지 않는다. 첫 설명은 주제 시점(새 용어로 넘어감)으로 보지 않는다. 첫 메시지라도 넘기는 말("알겠지?")이나 학생 질문에 대한 답이면 판단한다. "순서", "화면", "참고"처럼 고·서·면으로 끝나는 명사로 끝난 메시지는 설명이 끝난 것으로 본다. • 질문 문장은 만들지 않고, 궁금한 점 하나를 **이유 메모**로만 남긴다. 메모는 화면에 보이지 않는다. • 이미 손 든 이유나 이미 한 질문과 묻는 것이 같으면 들지 않는다. 같은 용어라도 묻는 것이 다르면 다른 질문이다. • 판정과 손 들기는 서로 격리된다. 손 들기가 어떤 이유로 실패해도 판정 결과는 돌려준다(`hand_raise: null`). 코드 오류는 `exception` 로그로 남긴다.
학생 반응 결정	• 선생님이 학생 질문에 답했고(`answer`) 학생이 다시 손 들지 않았을 때만 나온다. LLM을 부르지 않고, 페르소나별 고정 문장(`app/personas/reactions.json`)을 쓴다. • 그 수업의 몇 번째 답변인지에 따라 문장을 돌려 써서, 같은 문장이 연달아 나오지 않는다. 문장에는 수업 내용(용어·정답)이 들어가지 않는다. • 반응이 없는 경우: 답변이 이어지는 중(":", "그리고"…로 끝남), 다시 손 들었을 때, 손 들기 판단이 실패했을 때. • 짧은 답("응 맞아!")에도 반응한다. • Django는 말풍선으로만 보여 주고 messages에는 저장하지 않는다. 저장하면 다음 판정·노트에 섞인다.
질문받기 뒤 첫 메시지 결정	방금 보낸 메시지가 학생 질문 뒤 첫 선생님 메시지이면, `explanation`이고 `reply_to_id`가 없어도 답변으로 보고 학생 질문을 채운다. 그래서 학생 반응(`reaction`)과 손 들기 '답변' 시점(③)도 동작한다. 규칙 전체는 6장 "질문받기 뒤 첫 메시지"에 있다.
수용 기준	RNN 시나리오의 설명 ①~⑥은 해당 용어가 `explained`가 되고, 일부러 틀리게 한 ⑦(Truncated BPTT)은 `mentioned`에서 멈춘다. "스택과 큐" 수업에서 '큐'도 체크된다.

AI-4 학생 질문 만들기

입력	`persona`, `messages`(전체 대화), `reason`(손 든 이유)
출력	`content`: 학생 질문 한 개
규칙	• 페르소나 말투로, 손 든 이유에 궁금한 점이 여럿이어도 가장 막히는 하나만 묻는다. • 물음표는 하나만 쓴다. 두 개 이상이면 다시 부른다. 정답 후보를 나열하지 않는다("A예요, B예요?"). • 선생님이 쓰지 않은 기호 표기(예: `h_t`를 듣고 `h_{t-1}`을 만들기)가 있으면 다시 부른다. • 수업 대화나 페르소나의 '원래 아는 것'에 없는 **영문 용어**가 있으면 이유를 붙여 다시 부른다. 한글 낱말은 기록만 남긴다. • 위 검사는 마지막 시도 전까지만 적용한다. 마지막 시도에서도 남으면 질문을 그대로 돌려준다(버튼이 실패하지 않게).

AI-5 정리노트

입력	`title`, `persona`, `messages`(전체 대화), `terms`(체크리스트 용어 `[{term_id, name}]`, 체크리스트 순서대로. 선택, 기본 `[]` — v1.13 추가), `raised_reasons`(손 들었지만 선생님이 끝까지 누르지 않은 이유. AI-3에 보내는 것과 같은 값)
출력	`content`: Markdown 정리노트 (저장·내보내기용) `terms`: 화면용 구조 (v1.13 추가) `[{term_id, name, concept, questions: [{status, question, answer}]}]` • 요청한 용어가 순서대로 모두 온다. 설명하지 않은 용어는 `concept ""`, `questions []`. • 끝에 `term_id`가 `null`인 섹션('그 밖에 배운 내용')이 붙을 수 있다. • `concept`는 줄마다 줄바꿈(`\n`)으로 잇는다. • `status`: `answered` = 학생 AI의 질문 + 나의 답변 / `unseen` = 확인하지 않은 질문 (`answer: null`)

형식 결정	v1.13: 정리노트 화면 구성에 맞춰 용어별로 바꿈
	`# {title} 정리노트`
	`## 01. {용어 이름}`
	`### 개념` — 선생님 '설명' 메시지를 학생 AI가 페르소나 말투로 정리한 줄
	`### 학생 AI의 질문` — Q1. 학생 질문(원문) / 나의 답변: 선생님이 한 답(원문 그대로)
	`### 확인하지 않은 질문` — ① 질문받기를 누르지 않은 손 든 이유를 질문 한 문장으로 + ② 답 없이 끝난 학생 질문(원문)
	`## 02. {용어 이름} ...`
	`## 그 밖에 배운 내용` — 어느 용어에도 맞지 않는 내용 (있을 때만)

용어별로 나누기	• 개념 줄과 질문이 어느 용어에 대한 것인지는 AI가 `term_id`로 정한다. 여러 용어에 걸치면 가장 중심인 용어 하나에 넣는다. • 목록에 없는 `term_id`를 쓰면 이유를 붙여 다시 부르고, 그래도 안 되면 '그 밖에 배운 내용'에 넣는다. • `terms`를 보내지 않으면 섹션 하나(수업 제목)로 모두 나온다. • 학생 AI의 질문 → 확인하지 않은 질문 순서로 놓는다.

규칙	• **선생님이 말한 대로** 쓴다. 틀리게 말한 내용도 그대로 적고, 말하지 않은 내용은 맞더라도 쓰지 않는다. • 개념은 페르소나 말투로 쓴다: 초등 "~래요!", 고등 "제 말로 하면 …", 대학 개조식 "~임", 교수 "정의 — … (선생님 설명 기준)". 글투 표시가 있는 줄이 너무 적으면 다시 부른다(마지막 시도 제외). 모든 줄을 "선생님이 ~라고 하셨다"로 시작하지 않는다. • "맞아!,"처럼 느낌표 뒤에 붙은 쉼표는 코드가 지운다. • **나의 답변**은 선생님이 한 답을 원문 그대로 둔다. AI가 고쳐 쓰지 않는다. 한 질문에 답을 여러 메시지로 나눠 보냈으면 이어 붙인다. • 질문에 답하면서만 가르친 용어는 개념 칸에 "아래 질문과 답에서 배웠어요."를 쓴다. 개념으로 정리한 설명이 없는 용어는 "이 용어로 따로 정리한 내용은 없어요."를 쓴다. • 선생님 설명 문장을 빠뜨리지 않는다(노트 빠짐 검사). ◦ 코드가 설명 메시지의 각 문장(4글자 이상, 질문 제외)이 개념 줄에 담겼는지 글자 쌍(bigram) 겹침 50%로 확인한다. ◦ 빠진 문장이 있으면 그 문장을 알려 주며 다시 부른다(마지막 시도 제외). ◦ 의심스럽거나 짧은 문장("보통 다 잘 맞혀.")도 버리지 않는다. "오늘은 ~ 배울 거야" 같은 수업 예고 문장과 "오늘은 스택부터 할게.", "다음은 큐야."처럼 순서만 알리는 짧은 말은 내용이 아니므로 개념 줄로 쓰지 않고, 빠져도 다시 부르지 않는다. • 개념 줄마다 선생님 설명 메시지 원문 인용으로 검증하고, 선생님이 말하지 않은 숫자는 거부한다. • 본문 검사: 별도 호출이 개념 줄이 선생님 말만으로 나오는지 본다. 사실 여부가 아니라 '선생님이 그렇게 말했는가'를 보므로, 틀리게 배운 내용은 통과하고 사실대로 고쳐 적은 줄은 떨어진다. 떨어진 줄만 빼고 쓴다. • 답변은 `reply_to_id`로 질문과 짝 짓는다. 없으면 바로 앞의 답 안 된 질문과 짝 짓고, 질문을 못 찾으면 개념으로 돌린다. 학생 질문 뒤 첫 선생님 메시지는 `reply_to_id`가 없어도 답변으로 본다(6장 "질문받기 뒤 첫 메시지").
확인하지 않은 질문	• ① 학생이 손을 들었지만 선생님이 질문받기를 누르지 않은 것 (`raised_reasons`). 학생 AI가 페르소나 말투의 질문 한 문장(물음표 1개)으로 바꿔 쓴다. 대부분 여기에 해당한다. • ② 질문받기로 학생이 질문했는데, 그 뒤 선생님 메시지가 하나도 없이 수업이 끝난 것 (질문 원문). • 선생님이 답했지만 답이 틀리거나 부족한 질문은 '학생 AI의 질문'에 들어간다. 학생이 그 답을 듣고 다시 손 들었는데 선생님이 받지 않았으면, 그 이유가 ①로 들어간다. • 수업에 없던 영문 용어나 물음표가 1개가 아닌 문장은 다시 부른다. 끝내 못 쓰면 메모 원문을 쓴다.
수용 기준	틀리게 가르친 "순전파만 끊는다"와 짧은 설명이 노트에 그대로 있다. 가르치지 않은 용어는 없다. 4개 페르소나의 노트 말투가 서로 구분된다. 내용이 맞는 용어 아래에 들어간다. 받은 질문은 '학생 AI의 질문'에 나의 답변(원문)과 함께, 손 들고 묻지 못한 이유는 '확인하지 않은 질문'에 질문 꼴로 나온다. 실측(v1.13, 경사하강법 4명): 모두 200, 5.7~27.8초, LLM 호출 2~4번.

AI-6 출제 (평가·재시험)

입력	`kind` (`evaluation` / `retest`), `summary`, `terms`, `count`, `question_types`, 재시험이면 `focus_term_ids` (`explained` 가 아닌 용어), `exclude_questions` (이미 낸 문제)
출력	`questions: [{number, question_type, content, choices, correct_answer, explanation, term_id}]`
규칙	• 근거는 **요약정리뿐**이다(줄 번호와 인용으로 검증). 정확히 `count` 개를 내고, `number` 는 1부터 센다. • 코드가 출제 계획을 세운다: 유형 순서와 `term_id` 배정(재시험이면 `focus_term_ids` 만). • 객관식은 보기 4개, 정답은 0부터 센 보기 인덱스 문자열이다. 보기는 코드가 섞고, 오답 보기는 흔한 오개념으로 만든다. • 정답 보기는 요약정리 문장을 그대로 옮기지 않고, 다른 보기와 길이·문체를 맞춘다. 해설에는 줄 번호(L12)를 쓰지 않는다. • 이름 맞히기 문제와 질문에 정답이 드러난 문제는 거부한다(마지막 라운드만 허용). • 정답 독립 검사: 별도 호출이 정답이 요약정리로 뒷받침되는지 본다. • 재시험은 `exclude_questions` 와 같은 문제를 내지 않고, 핵심 정의를 묻는다. • 코드 `focus_term_ids` 에 `terms` 에 없는 id가 있거나, 지정한 용어가 모두 요약정리에 없으면 `422 VALIDATION_ERROR` 다. 전체 용어로 바꿔 내지 않는다.
Django 값 결정	평가 `count: 5` (학생 AI가 풂), 재시험 `count: 3` (사용자가 풂). 팀 결정 2026-11-11. 유형은 `question_types` `["multiple_choice","short_answer"]` 를 번갈아 배정한다. 평가 5문제 = 객관식 3 + 주관식 2, 재시험 3문제 = 객관식 2 + 주관식 1이다. 같은 유형을 두 번 넣을 수 없어서(최대 2개) 다른 비율은 AI 서버 수정이 필요하다. 백엔드는 지금 평가·재시험 모두 `EXAM_QUESTION_COUNT` 하나(10)를 쓰므로, 시험 종류마다 `count` 를 따로 보내야 한다 미반영.
문제 수를 줄인 이유	2026-11-11 실제 Claude로 용어 3개, 10문제를 내서 쟀다. • AI-6은 요약정리 줄을 근거로 문제를 낸다. 코드가 막는 중복은 똑같은 문장뿐이다. • 요약정리 12줄(경사 하강법 자료): 10문제가 모두 다른 줄을 근거로 썼다. 다만 비슷한 질문 1쌍이 있었다. • 요약정리 6줄(과적합·검증 데이터·정규화 자료): 한 줄을 3문제가 근거로 썼다. 같은 사실을 말만 바꿔 물어서, 사실 하나로 30점이 오갔다. • 결론: 근거 줄이 문제 수보다 적으면 같은 내용의 문제가 생긴다. 그래서 평가 5, 재시험 3으로 정했다. • 아직 확인 못 한 경우: 재시험 용어가 1개이고 요약 줄이 2개뿐이면 3문제 중 2문제가 같은 줄을 쓸 수 있다.

AI-7 학생 AI 문제 풀이

입력	`persona`, `messages` (전체 대화), `questions` (정답·해설 없이)
출력	`answers: [{number, response}]`
규칙	• **대화만 보고** 푼다. 근거 대화 인용이 없으면 `"모르겠어요. 수업에서 안 배웠어요."` 로 답한다. 틀리게 배운 내용은 틀리게 답한다. • 주관식 답은 페르소나 글투로 쓴다(`voices.json`). • 객관식 답은 보기 인덱스 숫자 하나만 쓴다("3"). 말투와 설명은 주관식에만 붙인다. 코드는 범위 안의 독립된 숫자가 정확히 하나일 때만 답으로 받는다. • 주관식 답에 수업에서 듣지 않은 숫자가 있으면 "모르겠어요"로 바꾼다. • 본문 검사(주관식·객관식): 선생님 말만으로 그 답을 낼 수 있는지 본다. 근거 없이 고른 객관식 답도 "모르겠어요"로 바꾼다. 정답 여부는 보지 않는다.
수용 기준	틀리게 가르친 문제는 오답, 가르치지 않은 문제는 "모르겠어요"다. 객관식 답이 문장이 되어 "모르겠어요"로 바뀌는 일이 없다(v1.3 실측, 4명 모두).

AI-8 주관식 채점

입력	`items: [{number, content, correct_answer, explanation, response}]`. 평가는 주관식 전부를 한 번에, 재시험은 한 문제씩 보낸다.
출력	`results: [{number, is_correct, reason}]`. `reason`은 화면에 보여 주지 않는다.
규칙	빈 답과 "모르겠어요"는 LLM 없이 오답이다. 모든 문항을 정확히 한 번씩 채점해야 한다. 빠진 번호와 중복 번호는 거부하고, 요청에 없는 번호는 무시한다. 요청에 평가·재시험 구분이 없으므로, 문항 수로 시간 예산을 정한다.

AI-9 성장보고서

입력	`title`, `persona`(이름·학년만), `messages`, `terms`(최종 상태), `exam`(`correct`, `total`, 문항별 결과)
출력	`strengths: [문장]`, `review_recommendations: [{term_id, text}]`
규칙	• 화면 구성은 잘한 점, 놓친 개념, 복습 추천이다(못한 점 칸은 없다). 보고서의 주인공은 선생님(사용자)의 설명이다. "학생이 잘 답했다"가 아니라 "~를 이렇게 설명해서 학생이 맞혔다"처럼 쓴다. • 잘한 점은 실제 대화 인용과 맞힌 문항으로 뒷받침되어야 한다. 지어낸 칭찬은 쓰지 않는다. 복습 추천 대상(놓친 용어)은 코드가 정한다. • `total` ≠ 문항 수, `correct` ≠ 맞힌 수, 또는 번호가 중복이면 `422 VALIDATION_ERROR`다. 코드 채점 안 된 문항(`is_correct: null`)이 있으면 `correct`가 "true 수 ~ true 수 + null 수" 사이인지 검사한다. • 점수, 점수 비교, 놓친 개념 목록은 Django가 계산한다.

6. 상태값과 데이터 규칙

Enum	값	비고
messages.sender / kind	`teacher`: `explanation`, `answer` · `student`: `greeting`, `question`	그 밖의 조합은 422
용어 status	`todo`, `mentioned`, `explained`	올라가기만 한다. `explained` = 체크
손 든 표시 status	`raised`, `asked`	Django 관리
exams.kind	`evaluation`, `retest`	평가는 학생 AI, 재시험은 사용자가 푼다
question_type	`multiple_choice`, `short_answer`	객관식 정답은 보기 인덱스 문자열
persona.code	`haneul`, `minjun`, `seoyeon`, `hyunwoo`	4-4 참고

- ID(`term_id`, 메시지 `id`, 문항 `number`)는 Django DB 값을 그대로 받아 그대로 돌려준다. LLM이 없는 ID를 내면 거부한다.
- AI-3에서 방금 보낸 메시지는 저장 전이라 `id`가 `null`일 수 있다.
- **질문받기 뒤 첫 메시지 = 답변** 결정 (팀 결정 2026-11-11: 질문받기를 했으면 선생님은 항상 답한다고 본다)
  - 학생 질문(`kind: question`) 뒤의 첫 선생님 메시지는 `kind`가 `explanation`이고 `reply_to_id`가 없어도, AI 서버가 그 질문의 답변(`answer`)으로 바꿔 읽는다.
  - 이 규칙은 대화를 받는 AI-3·4·5·7·9 모두에 적용된다. AI-3은 방금 보낸 메시지도 답변으로 보고 학생 질문을 채운다. 그래서 학생 반응(`reaction`)과 손 들기 '답변' 시점도 동작한다.
  - 질문이 여러 개 쌓였으면 가장 최근 질문부터 답한 것으로 본다. 정리노트 짝 짓기와 같은 순서다.
  - 답을 여러 메시지로 나누면 첫 메시지만 답변이 된다. 프론트가 `reply_to_id`를 보내면 모두 같은 질문에 묶인다.
  - 이유: 백엔드는 프론트가 `reply_to_id`를 보낼 때만 `answer`로 저장한다. 그런데 프론트는 아직 API 연결 전이라 보낼지 정해지지 않았다. 빠뜨리면 정리노트의 '학생 AI의 질문'이 비고 학생 반응도 안 나온다.
- 요약정리(`materials.summary`)와 용어 정의(`terms.definition`)는 프론트와 학생 AI에 절대 내려주지 않는다.

7. 품질·할루시네이션 요구사항

장치	막는 것	적용
Enum + 엄격한 JSON Schema	목록에 없는 값, 빠진 필드	모든 LLM 호출
RAG (BM25)	자료와 무관한 사전 지식	AI-2
원문 인용 검증	지어낸 근거. 인용은 8자 이상이어야 한다(원문이 짧으면 원문 전체, 또는 4글자 이상 문장 전체). 20자 미만은 글자 그대로 포함될 때만, 그 이상은 90% 일치도 인정하되 부정어(않·못·없·아니·안) 수가 다르면 거부한다.	정의, 요약, 관계, 판정, 노트, 출제, 풀이, 잘한 점
정의 독립 검사 (LLM)	인용은 진짜인데 다른 개념·식을 정의로 가져온 것	AI-1
본문 검사 (LLM)	인용은 진짜인데 본문이 다른 말을 하는 것	AI-2 요점·관계, AI-5, AI-7
정답 독립 검사 (LLM)	정답이 잘못 달린 문제	AI-6
숫자 대조 (코드)	근거에 없는 숫자	AI-1, AI-2, AI-5, AI-7
질문 검사 (코드)	수업에 없던 영문 용어, 새 기호 표기, 물음표 두 개 이상 (한글 낱말은 기록만)	AI-4
노트 빠짐 검사 (코드)	선생님 문장(특히 틀린 설명·짧은 설명)을 노트에서 빼먹는 것 (수업 예고 문장 제외)	AI-5
글투 검사 (코드)	페르소나 말투가 평범하게 수렴하는 것	AI-5
노트 용어 배정 검사 (코드)	목록에 없는 용어 번호에 넣는 것	AI-5
번호·ID 검증	없는 청크·메시지·줄·term_id	전부
코드가 정하는 것	LLM의 형식·계산 실수	Markdown 조립, 정리노트 용어별 구조, 출제 계획, 보기 섞기, 상태 올리기, 빈 답 채점, 손 들기 시점
정보 분리	학생 AI가 자료·정답을 보는 것	학생 프롬프트에는 페르소나·글투·수업 대화·손 든 이유 메모만 넣음

- **검사 AI의 출력 순서**: 정의 독립 검사, 본문 검사, 정답 독립 검사를 하는 AI는 판단 이유를 먼저 쓰고 그다음 결론(`supported`)을 낸다. 결론부터 쓰게 했을 때 이유에는 "문제 없음"이라고 쓰고도 탈락시키는 오판이 나왔고, 그 결과 틀리게 배운 ⑦이 노트에서 빠졌었다.
- **검사 호출 실패**: 정의 독립 검사나 본문 검사 호출 자체가 실패(오류·시간·형식)하면 그 검사만 건너뛰고 결과를 돌려준다 (DEBUG 기록). 검사 때문에 기능 전체가 502가 되지 않게 하기 위해서다.
- **재시도**: 형식이나 근거 검증에 실패하면 실패 이유를 붙여 한 번 다시 부른다. 응답이 max_tokens에서 잘리면 "더 짧게"를 붙여 다시 부른다(재시도 1회에 포함). 그래도 안 되면 `502 LLM_ERROR` 다. 떨어진 항목만 빼고 쓸 수 있는 작업은 남은 부분으로 응답한다.

8. 비기능 요구사항

8-1. 시간 예산과 실측

구분	AI 서버 예산	근거
실시간 (AI-3, AI-4, 1문항 AI-8)	25초	Django 타임아웃 30초보다 짧게

백그라운드 (그 밖)	170초	Django 타임아웃 180초보다 짧게

API	실측 (RNN 30쪽 PDF, v1.2) — 모두 예산 안
AI-1	약 75초 (정의 검사 탈락 1회 포함). 30쪽 중 29쪽을 이미지로 읽음 (약 13초)
AI-2	51~80초 (3회)
AI-3	3~6초
AI-4	4~6초 (다시 부르면 9~16초)
AI-5	16~23초(v1.2), 용어별 구조 5.7~27.8초(v1.13, 경사하강법 4명)
AI-6	28.5초(요약 12줄) / 58.8초(요약 6줄), 10문제 기준
AI-7	11.0초 / 12.9초, 10문제 기준
AI-8	3초
AI-9	8초

AI-6·AI-7의 10문제 값은 2026-11-11 실측이다. 5문제는 이보다 짧다. 모두 예산 170초 안이다.

자료 정리(AI-1 + AI-2)는 합쳐서 약 2~3분이 걸린다. 읽기 캐시 뒤로 AI-2는 이미지 자료에서 짧아지고(재측정 필요), AI-3는 손들기 시점이 아니면 판정만 해서 더 빨라졌다. 화면에 대기 상태가 필요하다(11장).

8-2. 에러

형식은 `{"error": {"code", "message", "request_id"}}` 이고, code는 명세의 다섯 가지만 쓴다. Django는 이 code를 `AI_ERROR` 로 바꾸되 `UNREADABLE_MATERIAL` 만 그대로 둔다.

code	HTTP	언제
`INVALID_INTERNAL_TOKEN`	401	토큰이 틀림, 또는 staging·prod인데 토큰 미설정
`VALIDATION_ERROR`	422	요청 형식 오류, 입력 상한 초과, 번호 중복·불일치, AI-6 집중 용어 없음
`UNREADABLE_MATERIAL`	422	AI-1·2에서 자료를 읽을 수 없음
`LLM_ERROR`	502	LLM 실패, 검증 재시도 실패, 응답 잘림 재시도 실패, 예상 못 한 오류
`LLM_TIMEOUT`	504	시간 예산 초과

8-3. 자료 읽기

- 지원 형식은 PDF(최대 50쪽), 텍스트, 코드, PNG·JPG·WebP 이미지다(WebP는 PNG로 바꿔 보냄. 이미지는 파일 앞 바이트로 판별). 업로드는 20MB, 이미지는 5MB까지다. LLM에 보내는 이미지는 긴 변 1,568px 이하로 줄인다.
- PDF는 쪽마다 상태를 코드로 판단한다.
  - **정상**: 글자만 뽑는다.
  - **그림 있음**: 글자 + `[그림: …]` 설명을 붙인다.
  - **글자 깨짐·스캔본**: 쪽 전체를 AI가 이미지로 읽는다. 4쪽씩 묶고 130 DPI로 보낸다.
- 위·아래 15% 가장자리에서 절반 이상의 쪽에 반복되는 머리말·꼬리말과 쪽 번호는 지운다.

8-4. 보안·운영

- `APP_ENV`: `local` (내 PC) / `staging` (팀 테스트 서버) / `prod` (배포).
  - 인터넷에 올리는 서버는 `prod` 로 두고 `AI_SERVER_TOKEN` 을 반드시 넣는다.
  - prod에서는 `/docs` (Swagger)가 꺼진다. 배포 서버에서 Swagger로 시험하려면 `staging` 을 쓴다.

- `AI_SERVER_TOKEN`은 Django와 같은 랜덤 문자열이다. 비교는 `hmac.compare_digest`로 한다.
- API 키와 `.env`는 깃·메신저·압축 파일로 공유하지 않는다(`.gitignore`에 포함). 키를 점검할 때는 값을 출력하지 않고 "있음/없음"만 보여 준다.
- 입력 상한
  - 메시지·문제·답 하나 10,000자, 대화 400개, 용어 50개, 문항 30개
  - 요약정리 200,000자, 이름·제목 200자, 페르소나 프롬프트 20,000자, 보기 10개
  - `X-Request-ID`가 128자 이하 영숫자·`._:-`가 아니면 새 UUID로 바꾼다.
- 업로드 크기 검사는 요청 본문을 다 받은 뒤에 한다(프레임워크 동작). 아주 큰 업로드를 받는 순간부터 막으려면 앞단 프록시에서 크기를 제한한다.
- LLM 동시 호출은 프로세스당 8개(`LLM_GLOBAL_CONCURRENCY`)다. 권장 구성은 프로세스 1개다(읽기 캐시도 프로세스마다 따로다).
- 설정값이 잘못되면(숫자 아님, 범위 밖, true/false 오타) 서버가 시작하지 않는다.

8-5. 로그

- 표준 출력에 JSON 한 줄씩 쓴다. 명세 이벤트는 `http_request`, `llm_call`, `llm_invalid_output`, `exception`이다.
- 사용자 내용(대화·자료)은 남기지 않는다. 예외는 `llm_invalid_output`의 `raw_output`(앞 2,000자)이고, `LOG_RAW_OUTPUT=false`면 길이만 남긴다.
- JSON 로거 이름은 `teachinglog`다. INFO 이상 이벤트는 명세의 네 가지만 쓴다.
- 검사 건너뜀(`claim_verification_skipped`, `definition_verification_skipped`, `exam_focus_terms_skipped`)과 디버그 이벤트(`llm_grounding_detail`, `grade_reasons`, `material_read_cache_hit` 등)는 DEBUG라서 `local`에서만 찍힌다.

9. 비용

- 모든 기능이 `claude-haiku-5-5` 하나를 쓴다.
- 30쪽 PDF 수업 1회(자료 준비 ~ 보고서)는 v1.0 실측 기준 수십 원 수준이었다. 그 뒤로 비용이 늘어난 것과 줄어든 것이 함께 있어서 재측정이 필요하다.
  - 늘어남: v1.1부터 검사 호출(정의, 본문, 정답 검사)
  - 줄어듦: v1.3부터 손 들기 호출(세 시점에만), v1.4부터 이미지 자료 다시 읽기(읽기 캐시)
- Haiku 5.5는 요청 한 번의 입력이 10만 토큰을 넘으면 단가가 5배가 된다. 가장 큰 요청(AI-2)도 1만 토큰대라 해당하지 않는다.
- 대회가 끝나면 API 키를 폐기하거나 서버를 내린다.

10. 설정과 실행

환경 변수는 세 가지만 필수다. 로컬은 `.env` 파일, 배포 서버(Render)는 대시보드의 환경 변수 칸에 넣는다. 나머지는 `app/config.py` 기본값을 쓰고, 바꿀 때만 한 줄을 추가한다. 숫자가 아니거나 범위를 벗어난 값, true/false 오타는 서버가 켜질 때 바로 오류를 낸다.

```
APP_ENV=prod
ANTHROPIC_API_KEY=sk-ant-...
AI_SERVER_TOKEN=(Django와 같은 랜덤 문자열)
```

자주 쓰는 선택 설정	기본값	뜻
`MIN_TERMS` / `MAX_TERMS`	3 / 10	용어 개수 범위. 이 안에서 AI가 핵심인 만큼 고른다(150자 미만 자료는 최소 1개)

`MAX_PDF_PAGES` / `MAX_UPLOAD_MB`	50 / 20	PDF 쪽수, 업로드 크기
`REQUIRE_TERM_NAME_FOR_EXPLAINED`	true	이름이 있어야 체크 (팀 규칙). 짝 개념은 한쪽 이름도 인정
`ENABLE_DEFINITION_VERIFICATION`	true	AI-1 정의 독립 검사
`ENABLE_CLAIM_VERIFICATION`	true	AI-2·AI-5·AI-7 본문 검사
`ENABLE_ANSWER_VERIFICATION`	true	AI-6 정답 독립 검사
`MAX_RAISED_HANDS`	0	쌓아 둘 손 든 표시 (0 = 제한 없음)
`ANTHROPIC_EFFORT_JUDGE`	high	AI-3 용어 판정의 생각 깊이 (나머지는 모델 기본 medium)
`LLM_MAX_OUTPUT_TOKENS`	16000	응답 최대 길이 (올리면 시간 예산 초과 위험)
`LLM_MIN_INTERVAL_SECONDS`	0	429가 잦으면 5~10
`LOG_RAW_OUTPUT`	true	검증 실패 로그에 원문을 남길지

실행 방법	명령
로컬	`pip install -r requirements.txt` → `python scripts/check_llm.py` (키·모델 연결 확인) → `uvicorn app.main:app --host 0.0.0.0 --port 8001 --no-access-log`
Render 배포	같은 명령에서 포트만 `--port $PORT` 로 바꾼다(Docker 없이). 환경 변수는 대시보드에 넣는다.
저장소 구성	배포 저장소에는 `app/`, `scripts/`, `requirements.txt`, `README.md`, PRD만 둔다. 테스트, 샘플, Docker 파일은 개발용 폴더에 따로 보관한다.

11. 화면 연동 요구사항

AI 서버 응답을 화면에 쓸 때 지켜야 할 것들이다. 디자인 대조 결과 고쳐야 하는 곳을 함께 적었다.

화면	요구사항
자료 제출	제목 입력 칸(필수, 1~100자)이 필요하다. 제출 후 09번을 폴링하는 대기 화면(약 2~3분)이 필요하고, `ready` 전에는 수업 시작을 막는다. 학생 카드 4장은 07번 응답으로 채운다(하늘, 민준, 서연, 정현우 교수님).
수업	• "질문 있어요!" 손 표시는 손 들게 만든 선생님 메시지(`after_message_id`) 바로 다음에 붙인다. 여러 개면 상단에 "질문 받기 (n)"을 둔다. • 첫 메시지는 페르소나별 고정 인사다. 내용을 아는 듯한 질문을 넣지 않는다. • 입력은 3,000자까지 받고, 응답 대기 중에는 로딩을 보여 주고 다시 보내기를 막는다. • "수업 저장"은 저장 API가 없다(메시지마다 바로 저장된다). "나가기"로 바꾸면 된다. • 선생님이 질문에 답하면 AI-3 `reaction`을 학생 말풍선으로 보여 준다. 저장하지 않는다.
정리노트	용어마다 개념 / 학생 AI의 질문(+나의 답변) / 확인하지 않은 질문. 끝에 '그 밖에 배운 내용'. 응답의 `terms`로 그린다. "학생 AI 평가 시작하기"는 진행 상태(17) 화면으로 가는 이동 버튼이다. 평가는 수업 종료 때 자동으로 시작된다.
학생 AI 평가	결정 평가 진행 화면을 따로 둔다. 학생 카드, 진행 표시("문제 풀이 중..." n/5), 문제 목록(푼 문제는 체크), "결과 보기" 버튼으로 구성한다. 17번은 푼 수/전체 수만 주고 문항 내용은 주지 않으므로, 문제 목록에 문항 문장을 보여 주려면 백엔드 응답에 문항이 필요하다. 참고: 학생 AI는 평가 문제를 AI-7 한 번으로 한꺼번에 푼다. 그래서 진행 상태의 `solved`는 한 문제씩 늘지 않고 0에서 전체 수로 한 번에 바뀐다. 문제마다 체크가 켜지는 모습이 필요하면 프론트에서 연출한다.

성장보고서	• "놓친 개념"은 문장이 아니라 `missed_terms` 목록(이름, 상태, 오답 수)으로 보여 준다. • 점수 비교는 같은 보고서 안의 "학생 AI 평가 → 내 재시험"만이다. 재시험 전이면 숨긴다. • "다시 설명하기"는 명세에 없다. 빼거나 "새 자료로 수업하기"로 바꾼다.
재시험	• 문제마다 제출하면 바로 정답·해설을 보여 준다(23번). 같은 문제는 다시 낼 수 없어서 "이전 문제"는 보기만 가능하다. • "답안 제출하기"는 결과 저장(24번)이다. • 주관식 입력 칸과 채점 로딩(몇 초)이 필요하다. 결과 화면의 "메모"는 만들어 주는 API가 없다.
공통	• 닉네임은 1~10자다(디자인은 "2~10자"). • 06-1 목록의 제목이 "정리노트"로 잘못 들어가 있다. • 사이드바의 "프로그램명"은 티칭로그로 바꾼다.

12. 검증과 수용 기준

12-1. 자동 테스트 (가짜 LLM, 개발용 폴더)

배포 저장소에서는 뺐고 개발용 폴더에 따로 보관한다. v1.3 기준으로 pytest 41개와 점검 스크립트 5묶음이 모두 통과했다.

종류	확인하는 것
API 규칙	인증, staging/prod 토큰, 에러 형식과 code, 요청 ID, 시간 초과, 429 대기, 업로드 상한, 입력 상한
명세 계약	에러 메시지 문구, 로그 이벤트·필드가 명세와 같음
단위	인용 검증, 별칭·한 글자 용어, 정답 노출, 요약 중복 제거, PDF 쪽 판단, 이름 맞히기 거부, 노트 용어 배정·빠짐 검사, 손 들기 시점, 객관식 답 파싱, 점수 범위
견고성	잘못된 JSON·필드 누락 뒤 복구, 검사 호출 실패 시 건너뜀, 판정·손 들기 예외 격리, 잘린 응답 토큰 기록, 설정 오타
흐름	AI-1 ~ AI-9 + 재시험 연결, 이미지 자료

> v1.4에서 들어간 읽기 캐시(AI-1 → AI-2 재사용, AI-2 성공 뒤 삭제, 15분 만료), 손 들기 명사 끝맺음 예외, AI-4 기호 표기 검사를 확인하는 테스트를 개발용 폴더에 추가하고, 전체를 다시 돌려야 한다. 정리노트 용어별 구조, 학생 반응, 글투 검사 테스트도 추가한다.

12-2. 실제 모델 확인 (RNN 시나리오)

자료 `chapter_5_2.pdf` (순환 신경망, 30쪽)로 설명 7개를 보낸다. ⑦은 일부러 틀리게 설명한다("역전파는 그대로 두고 순전파만 블록마다 끊는다"). Time RNN 계층, stateful, 마르코프 모델은 가르치지 않는다.

단계	기대 결과	결과
AI-1	핵심 용어만, 이름·정의가 자료와 일치	v1.3 통과 (10개 → 5개, 세부 용어 제외)
AI-2	요점이 자료와 일치	v1.2 통과 · 읽기 캐시 뒤 재측정 필요
AI-3	①~⑥ `explained`, ⑦ `mentioned`. 손 들기는 세 시점에만	통과
AI-4	손 든 이유에 맞는 질문 한 개, 물음표 하나	v1.3 통과 (4명 모두)
AI-5	틀린 설명·짧은 설명이 그대로, 안 가르친 용어 없음, 페르소나 말투 구분	v1.3 통과 (4명 모두)

AI-6	요약정리 근거, 정답 검사를 통과한 문제만	Swagger 통과
AI-7	틀리게 가르친 문제 오답, 안 가르친 문제 "모르겠어요", 객관식은 숫자	v1.3 통과 (4명 모두)
AI-8 · AI-9	채점 정확, 잘한 점이 실제 대화에 근거하고 주어가 선생님 설명	Swagger 통과 · 보고서 주어 수정 뒤 재확인 필요
재시험	놓친 용어 위주, 평가 문제와 겹치지 않음	v1.0 통과 · 이후 미확인
v1.4 추가	"스택과 큐" 자료에서 '큐'가 체크리스트에 들어가고 체크됨. 이미지 PDF에서 AI-2가 다시 읽지 않음	미확인
v1.5 추가	• 정리노트 3부분이 나오고 4명의 노트 말투가 구분됨 • 답변 뒤 `reaction`이 나옴	11/10 실제 모델로 4명 확인
v1.10 추가	• 짝 개념이 `A·B` 한 줄로 합쳐짐 • 한쪽 이름만 말해도 체크됨 • 틀린 설명은 `mentioned` • 짝 용어만으로 재시험 출제	11/11 실제 모델 통과 (경사 하강법 텍스트, AI-1·2·3·6)
v1.13	• 용어별로 나뉨 • 나의 답변이 원문 • 확인하지 않은 질문이 질문 꼴	11/11 실제 모델로 4명 통과

실측(v1.5): 경사하강법 샘플 수업에서 정리노트 10~21초, 꼬리 질문 0~2번.

실측(v1.14, 2026-11-11): 스택과 큐 수업으로 AI-1~AI-9와 재시험을 Django 순서대로 실제 Haiku로 돌려 17번 호출이 모두 성공. 정리노트 5~28초. 함정 수업 8번에서 틀리게 가르친 내용, 짧은 문장, 선생님 답변 원문이 8/8 그대로 남고, 안 가르친 내용이 새어 들어간 경우는 0번.

12-3. 제출 전 점검

- 개발용 폴더의 테스트가 전부 통과한다.
- `python scripts/check_llm.py`로 키와 모델 연결을 확인했다.
- 배포 서버 환경 변수: `APP_ENV=prod`, 토큰이 Django와 같고, 키가 깃에 없다.
- Django가 실시간 30초·백그라운드 180초 타임아웃을 지키고, AI-3에 방금 보낸 메시지를 대화 끝에 넣으며, 질문하지 않은 `raised_reasons`를 보내고, AI-5에 `terms`·`raised_reasons`를 보내고 응답 `terms`를 저장하며, AI-3 `reaction`을 저장하지 않으며, 페르소나를 다시 시드했다.
- Django 운영 서버의 Gunicorn `--timeout`이 실시간 AI 호출(30초)보다 길다(예: 60초). 기본값 30초면 AI 응답을 기다리는 중에 워커가 끊겨 정해진 에러 대신 끊긴 응답이 갈 수 있다. nginx `proxy_read_timeout`도 그보다 길게 둔다(기본 60초). 백그라운드 작업은 `db_worker`가 따로 돌려서 해당하지 않는다.

13. 남은 과제

우선	위치 · 담당	내용

높음	AI · 실제 모델	v1.4 변경을 실제 Claude로 확인한다. • "스택과 큐" 자료에서 한 글자 용어가 체크되는지 • 이미지 PDF에서 읽기 캐시가 쓰이는지와 AI-2 시간 • 보고서 주어, 재시험 • 놓친 용어가 1~2개인 수업에서 재시험 10문제가 겹치지 않고 170초 안에 나오는지 — 해결(v1.12): 재시험을 3문제로 정함
높음	백엔드	Django가 고쳐야 할 것 8가지(아래 표): AI-5 요청에 `terms`·`raised_reasons` 추가, AI-5 응답 `terms` 저장, AI-3 `reaction` 표시(저장 안 함), 평가 5·재시험 3 `count` 나누기, 페르소나 4명 새 데이터 마이그레이션, `hyunwoo` 첫 인사, WebP 업로드 허용(선택). ERD "3명 고정" 설명과 07번 응답도 4명에 맞춘다.
중간	AI · 개발용 폴더	읽기 캐시, 명사 끝맺음 예외, 기호 표기 검사 테스트를 추가하고 전체를 다시 돌린다.
중간	명세서	코드 동작을 기능명세서·API 명세서에 반영한다. • 용어 체크 규칙, 손 들기 시점, 핵심 용어 3~10개, 페르소나 4명, 정리노트 용어별 구조, `reaction`(AI-3), AI-5 `terms`(요청·응답), `raised_reasons` • AI-6 422, AI-9 범위 검사, 응답 잘림 재시도, prod `/docs` 꺼짐, 입력 상한 • 읽기 캐시(원본 폐기 원칙에 "메모리 최대 15분"을 명시)
중간	프론트·백엔드	11장 화면 연동 요구사항을 반영한다.
낮음	AI	비용과 시간을 재측정한다(수업 1회, 평가 5문제 기준).
낮음	AI	노트의 다른 수업 도입 문장 때문에 다시 불리는 경우를 줄인다.

Django가 고쳐야 할 것 (2026-11-11, Django `dev` 코드와 비교)

지금 Django 코드로 불러도 에러는 나지 않는다(새 필드는 모두 선택). 하지만 아래를 고치지 않으면 기능이 빠지거나 다르게 동작한다.

#	고칠 곳	할 일	안 하면 생기는 일	필요 여부
1	`lessons/ai.py` `note_payload`	AI-5 요청에 `terms` 추가	노트가 용어별로 나뉘지 않고, 모든 내용이 섹션 하나(수업 제목)에 섞여 나옴	필수
2	`lessons/ai.py` `note_payload`	AI-5 요청에 `raised_reasons` 추가	학생이 손 들었는데 선생님이 받지 않은 궁금증이 노트의 '확인하지 않은 질문'에서 빠짐	필수
3	`lessons/models.py` `Note`, `parse_note`	AI-5 응답의 `terms`(용어별 구조)도 저장	Markdown(`content`) 하나만 남아서 화면이 용어별 박스로 그릴 수 없음	화면을 용어별로 그리면 필수
4	AI-3 응답 처리 (`parse_analyze_result`)	`reaction`을 읽어 학생 말풍선으로 내보내기 (messages에는 저장 안 함)	선생님이 학생 질문에 답해도 학생이 아무 반응을 하지 않음	필수
5	`exams/views.py` `EXAM_QUESTION_COUNT`	평가는 `count: 5`, 재시험은 `count: 3`으로 따로 보내기	둘 다 같은 수(지금 10)로 나옴. 요약정리 줄이 적으면 같은 사실을 여러 문제가 말만 바꿔 물어서 점수가 왜곡되고, 시간도 더 걸림	필수
6	`personas` 새 데이터 마이그레이션	`app/personas/personas_seed.json`의 4명으로 시드 (`hyunwoo` 추가, 나머지 3명 `system_prompt`·설명 교체)	정현우 교수님을 고를 수 없음. 하늘·민준·서연은 Django의 초안 프롬프트로 돌아서 아는 범위·손 드는 때·질문 방식이 설계와 다르게 나옴	필수

7	`lessons/greetings.py`	`hyunwoo` 첫 인사 추가	정현우 교수님도 기본 인사("선생님 안녕하세요! … 잘 들을게요.")를 함	6번과 함께
8	`materials/views.py` `ALLOWED_EXTS · IMAGE_EXTS`	`webp` 추가	WebP 이미지를 올릴 수 없음 (AI 서버는 이미 받을 수 있음)	선택

- 참고: 백엔드는 출제 뒤 `question_count`를 실제로 받은 문제 수로 저장하고, 점수도 그 수로 계산한다.
- 그대로 두어도 되는 것: `reply_to_id`는 지금처럼 받을 때만 `answer`로 저장해도 된다(6장 "질문받기 뒤 첫 메시지"). 프론트가 보내면 질문이 여러 개일 때 짝이 더 정확해진다.

v1.3 ~ v1.12에서 해결된 것은 다음과 같다.

- 한 글자 용어 매칭, 손 들기 ④ 제거, "순서·화면" 같은 명사 끝맺음 오판
- 정의 검사 실패 시 전체 실패, 잘린 호출 토큰 누락, 명세에 없던 WARNING 로그 3개
- 서비스명 표기, AI-1·AI-2 자료 두 번 읽기, 정리노트 구성 고정(v1.5) → 용어별 구조(v1.13)
- 짝 개념 한 줄로 합치기(v1.10)

14. 위험과 팀 결정 사항

14-1. 남는 위험

위험	설명과 대응
인용은 진짜, 해석이 틀림	정의 독립 검사와 본문 검사로 막는다. 다만 검사도 LLM 판단이라 놓칠 수 있고, 검사 호출이 실패하면 건너뛴다.
재시험 문제 수	결정: 3문제 (팀 결정 2026-11-11). 백엔드에 재시험 전용 설정 필요. 이유와 실측은 AI-6 "문제 수를 줄인 이유"에 있다.
핵심 용어 판단	"핵심"은 LLM 판단이라 자료마다 개수가 달라진다. 서로 짝인 개념(전역 최솟값·지역 최솟값)을 둘 다 고르는 경우가 있다. 짝인 개념은 체크리스트 한 줄(`A·B`)로 합친다. 반영했고 실제 모델로 확인했다(자료 1개). 자료에 따라 짝을 놓치거나 짝이 아닌 개념을 합칠 수 있다.
검사 AI의 오판	너무 엄격하면 용어·요점이 줄고, 틀리게 배운 내용을 사실 기준으로 떨어뜨리면 수용 기준이 깨진다. 이유를 먼저 쓰게 고친 뒤 AI-5는 3/3 통과했다. 같은 수정을 한 AI-1·AI-2·AI-7 검사는 실제 모델로 다시 확인하지 않았다.
숫자	코드는 원문에 **없는** 숫자만 잡는다. 다른 대상에 붙인 숫자, 한 자리 정수, 한글 수, 단위 변경은 LLM 검사에 기댄다. AI-3·AI-6·AI-8·AI-9에는 숫자 대조가 없다.
메시지 하나만 판정	이름은 앞 메시지에서 말하고 설명은 다음 메시지에서 하면 체크되지 않는다(명세 규칙).
손 들기 주제 전환	새 용어는 체크리스트 이름으로 알아낸다. 체크리스트에 없는 주제로 넘어가면 "넘어가자", "정리하면" 같은 말이 있을 때만 잡힌다.
이미지로 읽은 글	원본 글자와 대조할 수 없어서 잘못 읽은 글자는 검증으로 잡히지 않는다.
AI-4 질문 모양	물음표는 하나지만 "그리고"로 두 가지를 묻는 질문이 가끔 나온다. 한글 추측 낱말은 프롬프트 지시에만 기댄다.
읽기 캐시	서버가 재시작하거나 잠들면 사라져서 다시 읽는다(비용만 듦). 여러 프로세스로 띄우면 공유되지 않는다. AI-1 뒤 15분 안에 AI-2가 오지 않아도 다시 읽는다.

14-2. 팀이 정할 것

항목	선택지

정리노트 구성 결정	• 체크리스트 용어마다 개념 · 학생 AI의 질문 · 확인하지 않은 질문으로 바꿨다(v1.13, 정리노트 화면 구성에 맞춤). 예시 칸은 따로 두지 않는다. • 용어 카드 화면처럼 용어별 구조를 주는 안은 AI-5 응답 `terms`로 해결됐다(v1.13).
재시험 문제 수 결정	결정: 3문제 (팀 결정 2026-11-11). 백엔드에 재시험 전용 설정 필요
운영 로그의 `raw_output`	명세대로 유지(접근·보관 제한) / `LOG_RAW_OUTPUT=false` + 로그 규칙 수정
AI-6 집중 용어가 없을 때	422(현재 코드) / 전체 용어로 출제(예전)
AI-4 마지막 시도 처리	질문을 돌려줌(현재) / `502`로 거부

이미 정한 것

- explained 기준: 용어 언급 + 간단한 설명, 이름 필수 (짝 개념 `A·B`는 한쪽 이름도 인정)
- 손 들기는 세 시점에만, 페르소나 4명
- 성장보고서 구성: 잘한 점 · 놓친 개념 · 복습 추천
- 판정(AI-3 용어 판정)만 effort high, 나머지는 기본(medium)
- 모델은 Haiku 5.5 하나, 배포는 Render(Docker 없음), 서비스명은 티칭로그
- PPT·DOCX는 백엔드가 PDF로 변환
- 학생 반응은 LLM 없이 고정 문장으로, 답변 뒤에만 보여 준다. 다시 손 들면 반응하지 않는다.
- 꼬리 질문 횟수는 제한하지 않는다.
- 페르소나 글투는 AI 서버가 관리한다. Django에서 새로 만든 페르소나는 기본 말투로 쓴다.
- 평가 5문제, 재시험 3문제
- 평가 진행 화면은 따로 둔다.
- 업로드는 20MB, PDF 50쪽까지(PPT·DOCX는 PDF로 바꾼 뒤 기준)
- 짝인 개념은 체크리스트 한 줄로 합친다.

15. 용어

용어	뜻
용어 체크리스트	자료에서 AI가 고른 핵심 용어 3~10개. 수업 목표이자 판정·출제 단위다.
체크 (explained)	용어 이름을 말하고 간단하게라도 맞게 설명한 상태
요약정리	용어별 정의, 요점, 관계를 담은 Markdown. 출제의 유일한 근거다.
정리노트	학생 AI가 대화만 보고 쓴 노트. 틀리게 배운 내용도 그대로 들어간다. 체크리스트 용어마다 개념 · 학생 AI의 질문 · 확인하지 않은 질문으로 나뉜다.
손 든 표시	학생 AI가 궁금한 점이 생겼다는 표시. 누르면 질문이 된다.
손 들기 시점	코드가 정하는 판단 시점: 새 용어로 넘어감, 넘기는 말, 답변 직후
노트 빠짐 검사	선생님이 한 설명 문장이 정리노트에 빠지지 않았는지 코드로 확인하는 것
학생 반응	선생님이 질문에 답했을 때 학생이 보이는 고정 문장. 저장하지 않는다.
확인하지 않은 질문	손 들었지만 질문받기를 누르지 않은 이유를 질문 꼴로 바꾼 것과, 질문 뒤 선생님 메시지 없이 수업이 끝난 질문
나의 답변	학생 AI의 질문에 선생님이 한 답. 원문 그대로 둔다
글투	페르소나별 말하는 방식 (`voices.json`)
읽기 캐시	AI-1에서 AI가 이미지로 읽은 자료 글자를 AI-2가 다시 쓰도록 메모리에 잠깐 두는 것

원문 인용 검증	LLM이 낸 근거 문장이 자료, 대화, 요약정리에 실제로 있는지 코드로 확인하는 것
정의 독립 검사	만든 AI와 별개의 호출이, 정의가 근거와 같은 말인지와 그 용어 자체의 정의인지 다시 확인하는 것
본문 검사	만든 AI와 별개의 호출이, 문장이 주어진 근거만으로 나오는지 다시 확인하는 것
시간 예산	요청 하나가 LLM을 부르는 데 쓸 수 있는 최대 시간